Dunder Mifflin - Application Requirements Specification

Name:	Amogh Doddagadduvalli Prasanna
Email:	amoghdp@umd.edu
UMD directory ID:	amoghdp
Application name:	Dunder Mifflin

Table of Contents

Dunder Mifflin - Functional requirements ---------- ***3***

1. User Interfaces ---------- ***6***
2. Backend API ---------- ***6***
3. Database interface (PostgreSQL) ---------- ***6***
4. Misuses ---------- ***8***
5. STRIDE threat modeling ---------- ***12***

1. Introduction

The Dunder Mifflin Inventory System is designed to support product catalog management, stock tracking, and role-based user operations.

2. Functional Requirements

FR-001 - Add New Product

- **Use Case Reference:** UC-01 Add New Product

- **Trigger:** Warehouse Employee selects “Add Product” option from the Dashboard.

- **Preconditions:**
  - User is authenticated and has role = Warehouse Employee.
  - Mandatory product fields (SKU, name, quantity) available.

- **Success Flow:**
  - Employee navigates to Add Product page.
  - Enters product details and submits form.
  - System validates input fields.
  - Business Logic Layer inserts product into DB via Data Access Layer.
  - System confirms success with message.

- **Alternative Flows:**
  - Admin adds product via API call (batch import).
  - Pre-populated form for re-adding discontinued products.

- **Postconditions:**
  - New product record persisted in DB.
  - Entry is logged in inventory_moves ledger.

- **Error Flows:**
  - Invalid input → error message with highlighted fields.
  - Duplicate SKU → system rejects insert and prompts user.
  - DB write failure → rollback and error message.

FR-002 - View Current Stock Levels

- **Use Case Reference:** UC-02 View Inventory

- **Trigger:** Sales Associate selects “View Inventory” from Dashboard.

- **Preconditions:**
  - User authenticated as Sales Associate or higher.

- **Success Flow:**
  - User navigates to Inventory List page.
  - System retrieves list of products from DB.
  - Results are displayed with stock levels and filtering options.

- **Alternative Flows:**
  - API call returns JSON for integration with sales system.

- **Postconditions:**
  - Product catalog is displayed or returned as JSON.

- **Error Flows:**
  - DB unavailable → system shows fallback message: *“Inventory temporarily unavailable”*.
  - Empty dataset → system shows *“No products available”*.

FR-003 - Update Existing Product Quantity

- **Use Case Reference:** UC-03 Update Product Quantity

- **Trigger:** Warehouse Manager selects product and submits new quantity.

- **Preconditions:**
  - User authenticated as WarehouseManager.
  - Product must exist in DB.

- **Success Flow:**

    - Manager opens Update Product page.
    - Enters new quantity and submits.
    - System validates request.
    - Business Logic updates qty_on_hand and logs move.
    - Confirmation displayed.

- **Postconditions:**
    - Updated quantity stored in DB.
    - Entry logged in inventory_moves.

- **Error Flows:**
    - Negative quantity → validation error.
    - Concurrent update conflict → system retries or shows warning.
    - DB update failure → rollback + error message.

FR-004 - Authentication & Authorization

- **Use Case Reference:** Applies to all use cases.

- **Trigger:** User attempts login or protected action.

- **Preconditions:** User account exists in users' table.

- **Success Flow:**
    - JWT issued after successful login.
    - Middleware validates JWT on each request.
    - RoleGuardMiddleware ensures role-based access.

- **Error Flows:**
    - Invalid JWT → *401 Unauthorized*.
    - Expired JWT → prompt re-login.

3. External Interface Requirements

a. User Interfaces

- Users (Sales Associate, Warehouse Employee, Manager) interact with the application through a **modern web browser**.
- The UI is responsive (HTML/CSS/JS, extendable to React + Tailwind).
- Core user flows:
    - Sales Associates can log in, search/view products, and check stock levels.
    - Warehouse Employees can add products and update inventory.
    - Managers can view reports, update quantities, and audit inventory moves.

b. Backend API (Go)

- The frontend communicates with the backend via **RESTful APIs**.
- All requests and responses use **JSON format**.
- Key endpoints include:
    - POST /login → Authenticate users and issue JWT.
    - POST /products → Add a new product.
    - GET /inventory → Retrieve current stock levels.
    - PATCH /products/:id/quantity → Update product quantity.
    - GET /inventory/:id/moves → Fetch product transaction history.

c. Database Interface (PostgreSQL)

- Stores information about **users, products, and inventory moves**.
- Accessed only through the backend's Data Access Layer.
- Enforces integrity through migrations, constraints, and triggers.

d. Data Formats

- **REST API**: JSON (UTF-8 encoded).
- **Identifiers**: UUID strings (primary keys).
- **Monetary values**: Stored as integer cents (price - cents).
- **Date/Time**: ISO 8601 strings (UTC).
- **Ledger entries**: Immutable rows in inventory_moves.

e. External Services

- None.
- Future consideration: integration with external ordering or sales systems (via secure APIs).

3.2. Completeness Considerations

- Product lifecycle support (add, view, update).
- Role-based access (Employee, Associate, Manager).
- Error handling validation, DB failures, and concurrency.
- Ledger (inventory_moves) ensures auditability.
- Future extensibility: product removal, bulk imports, order integration.

4. Misuse Cases

This section documents potential security threats by analyzing how malicious actors might attempt to misuse the Dunder Mifflin Inventory System. Each misuse case identifies the attacker, their goal, attack method, and potential impact.

4.1 MUC-001: Unauthorized Access Through Credential Theft

Attacker: External malicious actor or disgruntled former employee

Goal: Gain unauthorized access to the inventory system to view sensitive data, manipulate inventory records, or disrupt operations

Method:

- Phishing attacks targeting employees to steal login credentials
- Brute force attacks against weak passwords
- Session hijacking through network sniffing if HTTPS is not enforced
- Credential stuffing using passwords leaked from other breaches

Impact:

- Exposure of sensitive inventory and pricing data
- Unauthorized modification of inventory quantities leading to stock discrepancies
- Creation or deletion of products causing operational chaos
- Loss of customer trust and potential financial losses

4.2 MUC-002: SQL Injection Attack

Attacker: External hacker with knowledge of web application vulnerabilities

Goal: Exploit database vulnerabilities to extract, modify, or delete data, or gain elevated privileges

Method:

- Inject malicious SQL code through search fields, product name inputs, or filter parameters
- Manipulate API parameters in product update requests
- Exploit poorly sanitized input in authentication queries

Impact:

- Complete database compromise including user credentials and audit logs
- Mass deletion of inventory records
- Elevation of privileges to administrator level
- System downtime and data recovery costs

4.3 MUC-003: Privilege Escalation

Attacker: Authorized warehouse staff user seeking unauthorized administrative access

Goal: Gain manager or administrator privileges to access restricted functions like adding products, managing users, or viewing sensitive reports

Method:

- Manipulate API requests to bypass role-based access controls
- Exploit session management vulnerabilities to impersonate higher-privileged users
- Modify cookies or tokens to change role parameters
- Exploit insecure direct object references to access admin-only endpoints

Impact:

- Unauthorized creation or deletion of products and users
- Access to confidential pricing and business intelligence data
- Violation of separation of duties and audit trail integrity
- Potential insider fraud and inventory manipulation

4.4 MUC-004: Inventory Manipulation for Fraud

Attacker: Insider employee (warehouse staff or manager)

Goal: Manipulate inventory quantities to cover theft, create ghost inventory, or facilitate fraudulent transactions

Method:

- Make unauthorized quantity adjustments to hide stolen products
- Create fake IN movements to inflate inventory counts
- Delete or modify audit log entries if access controls are weak
- Collude with another user to perform and approve fraudulent transactions

Impact:

- Financial losses due to undetected theft
- Inaccurate inventory counts affecting business operations and decisions
- Compromised audit trail making fraud detection difficult
- Loss of inventory accuracy and management trust

4.5 MUC-005: Denial of Service Attack

Attacker: External attacker, competitor, or disgruntled individual

Goal: Make the inventory system unavailable to legitimate users, disrupting warehouse operations

Method:

- Flood the system with excessive API requests (DDoS attack)
- Submit extremely large product descriptions or search queries to exhaust resources
- Exploit unoptimized database queries to cause performance degradation
- Create thousands of fake user sessions to exhaust server memory

Impact:

- System downtime preventing inventory tracking and updates
- Warehouse operations disruption and inability to fulfill orders
- Customer dissatisfaction due to delayed shipments
- Financial losses from lost productivity and potential SLA violations

4.6 MUC-006: Cross-Site Scripting (XSS) Attack

Attacker: External attacker or malicious insider

Goal: Inject malicious JavaScript code to steal session tokens, redirect users, or perform actions on behalf of victims

Method:

- Inject malicious scripts into product names or descriptions
- Store XSS payloads that execute when other users view the inventory
- Exploit search or filter functionality to execute scripts

Impact:

- Session hijacking and account takeover
- Theft of sensitive information displayed on screen
- Unauthorized actions performed using victim's credentials
- Malware distribution or phishing attacks against users

4.7 MUC-007: Audit Log Tampering

Attacker: Insider with database access or administrator who has committed fraud

Goal: Delete or modify audit logs to cover tracks after committing fraudulent activities or policy violations

Method:

- Direct database access to delete movement_history records
- Exploit insufficient access controls on audit log tables
- Modify timestamps or user IDs to misattribute actions

Impact:

- Inability to detect and investigate fraud or policy violations
- Loss of non-repudiation and accountability
- Compliance violations with audit and regulatory requirements
- Erosion of trust in system integrity

5. STRIDE Threat Modeling

This section applies to the STRIDE framework (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to systematically analyze security threats across all components of the Dunder Mifflin Inventory

System.

5.1 Spoofing Identity Threats

Definition: Pretending to be someone or something else to gain unauthorized access

Threats Identified:

- **S-01: Weak Password Authentication -** Attackers can guess or brute force weak passwords to impersonate legitimate users
- **S-02: Session Token Theft -** Session cookies can be stolen through XSS or network interception if not properly secured
- **S-03: Missing Multi-Factor Authentication -** Single-factor authentication makes credential theft easier
- **S-04: API Request Forgery -** Attackers can craft API requests pretending to be the legitimate application

Mitigations:

- Enforce strong password policies (minimum 12 characters, complexity requirements)
- Implement account lockout after failed login attempts
- Use secure, HTTPOnly, SameSite session cookies
- Implement CSRF tokens for state-changing operations
- Consider implementing MFA for administrator accounts

5.2 Tampering with Data Threats

Definition: Malicious modification of data in transit or at rest

Threats Identified:

- **T-01: SQL Injection -** Malicious SQL code can modify or delete database records
- **T-02: API Parameter Manipulation -** Attackers can modify request parameters to change quantities, prices, or product information
- **T-03: Mass Assignment Vulnerability -** Unprotected fields in update operations could allow modification of read-only attributes
- **T-04: Audit Log Manipulation -** Direct database access or weak controls allow modification of movement history

- **T-05: Client-Side Data Manipulation -** JavaScript modifications to bypass client-side validation

Mitigations:

- Use parameterized queries and prepared statements for all database operations
- Implement strict input validation on all API endpoints
- Explicitly whitelist allowed fields for update operations
- Make audit logs append-only with restricted database permissions
- Implement server-side validation independent of client-side checks
- Use database triggers to prevent unauthorized modifications to critical tables

5.3 Repudiation Threats

Definition: Users deny performing actions without a way to prove otherwise

Threats Identified:

- **R-01: Incomplete Audit Logging -** Not all actions are logged, allowing users to deny certain operations
- **R-02: Missing User Attribution -** Actions are logged without proper user identification
- **R-03: Shared Account Usage -** Multiple users sharing credentials prevent individual accountability
- **R-04: Log Tampering -** Users with elevated access can modify or delete their own log entries

Mitigations:

- Log all security-relevant events (login, logout, data modifications, failed access attempts)
- Include timestamp, user ID, action type, affected resources, and outcome in logs
- Enforce unique user accounts (no shared credentials policy)
- Implement write-only audit logs with database-level access controls
- Consider log forwarding to external SIEM system for tamper-proof storage

5.4 Information Disclosure Threats

Definition: Exposure of information to unauthorized individuals

Threats Identified:

- **I-01: Insecure Direct Object References -** Users can access other products or records by manipulating IDs in URLs
- **I-02: Excessive Error Information -** Detailed error messages expose database structure, file paths, or technical details
- **I-03: Unencrypted Network Traffic -** Sensitive data transmitted over HTTP can be intercepted
- **I-04: Weak Access Controls -** Staff users can view pricing or administrative data intended only for managers
- **I-05: Password Storage Weakness -** Weak hashing algorithms allow password recovery if database is compromised
- **I-06: Session Information Exposure -** Session tokens exposed through logs or URLs

Mitigations:

- Implement role-based access control on all API endpoints and UI elements
- Use generic error messages for users; log detailed errors server-side only
- Enforce HTTPS for all communications
- Validate user permissions before returning any data
- Use bcrypt with appropriate cost factor (minimum 10) for password hashing
- Avoid including sensitive information in URLs or logs

5.5 Denial of Service Threats

Definition: Making resources unavailable to legitimate users

Threats Identified:

- **D-01: API Request Flooding -** Overwhelming the server with excessive requests
- **D-02: Resource Exhaustion -** Submitting extremely large payloads or complex queries to consume server resources
- **D-03: Database Connection Pool Exhaustion -** Creating many concurrent connections to exhaust available database connections

- **D-04: Login Attempt Flooding -** Repeated failed login attempts consuming authentication service resources
- **D-05: Algorithmic Complexity Attack -** Crafted inputs that trigger worst-case performance in sorting or search algorithms

Mitigations:

- Implement rate limiting on API endpoints (e.g., 100 requests per minute per IP)
- Set maximum payload sizes and request timeouts
- Use connection pooling with maximum connection limits
- Implement progressive delays or CAPTCHA after failed login attempts
- Optimize database queries and add appropriate indexes
- Consider implementing request queuing and load balancing for high availability

5.6 Elevation of Privilege Threats

Definition: Gaining unauthorized access to higher-level permissions or capabilities

Threats Identified:

- **E-01: Horizontal Privilege Escalation -** Staff users accessing functions intended for managers by manipulating requests
- **E-02: Vertical Privilege Escalation -** Normal users gaining administrator access through session manipulation or exploits
- **E-03: Role Parameter Tampering -** Modifying role values in cookies, tokens, or API requests
- **E-04: SQL Injection for Privilege Escalation -** Using SQL injection to modify user role in the database
- **E-05: Path Traversal to Admin Functions -** Accessing admin-only URLs or API endpoints without proper authorization checks

Mitigations:

- Implement server-side authorization checks on every request
- Store role information server-side in session, not in client-accessible tokens
- Use middleware to enforce role-based access control on all routes
- Apply principle of least privilege - deny by default, allow explicitly
- Use parameterized queries to prevent SQL injection
- Conduct regular code reviews focusing on authorization logic

6. Non-Functional Requirements

6.1 Performance Requirements

- Page load time shall not exceed 2 seconds under normal load
- System shall support up to 50 concurrent users
- Database queries shall return results within 1 second
- API response time shall not exceed 500ms for standard operations

6.2 Security Requirements

6.2.1 Authentication and Authorization

- **SR-AUTH-01:** All passwords shall be hashed using bcrypt with minimum cost factor of 12
- **SR-AUTH-02:** Password policy shall enforce minimum 12 characters with at least one uppercase, lowercase, number, and special character
- **SR-AUTH-03:** Account lockout shall occur after 5 consecutive failed login attempts for 30 minutes
- **SR-AUTH-04:** Session timeout shall occur after 30 minutes of inactivity
- **SR-AUTH-05:** All session cookies shall use HTTPOnly, Secure, and SameSite=Strict flags
- **SR-AUTH-06:** Role-based access control shall be enforced on every API endpoint and UI component
- **SR-AUTH-07:** Authorization checks shall be performed server-side and shall not rely on client-side validation

6.2.2 Input Validation Requirements

- **SR-INPUT-01:** All user inputs shall be validated on the server side using whitelist validation

- **SR-INPUT-02:** Product names shall be validated to prevent special characters used in XSS attacks (< > ' " &)
- **SR-INPUT-03:** Numeric fields (quantity, price) shall be validated for type, range, and format
- **SR-INPUT-04:** Maximum input lengths shall be enforced (100 chars for names, 500 for descriptions, 200 for notes)
- **SR-INPUT-05:** All SQL queries shall use parameterized statements or prepared statements to prevent SQL injection
- **SR-INPUT-06:** API request payloads shall have maximum size limits (1MB for standard requests)
- **SR-INPUT-07:** File upload functionality (if implemented) shall validate file types, sizes, and content

6.2.3 Data Protection Requirements

- **SR-DATA-01:** All data transmission shall use HTTPS/TLS 1.2 or higher encryption
- **SR-DATA-02:** Sensitive data (passwords, session tokens) shall never be logged
- **SR-DATA-03:** Database backups shall be encrypted at rest
- **SR-DATA-04:** Database connection strings and API keys shall be stored in environment variables, not in source code
- **SR-DATA-05:** Users shall only access data they are authorized to view based on their role
- **SR-DATA-06:** Audit logs shall be stored in append-only tables with restricted DELETE permissions

6.2.4 Error Handling and Logging Requirements

- **SR-ERROR-01:** Error messages displayed to users shall not reveal sensitive system information
- **SR-ERROR-02:** Detailed error information shall be logged server-side for debugging purposes
- **SR-ERROR-03:** Generic error messages shall be used for authentication failures (e.g., 'Invalid username or password' instead of 'Username not found')
- **SR-ERROR-04:** Stack traces and database error details shall never be exposed to end users
- **SR-ERROR-05:** HTTP error codes shall be used appropriately (401 for unauthorized, 403 for forbidden, 404 for not found)

6.2.5 Audit and Monitoring Requirements

- **SR-AUDIT-01:** All authentication events shall be logged (successful and failed login attempts, logout)
- **SR-AUDIT-02:** All data modification operations shall be logged in movement_history table with user ID, timestamp, before/after values
- **SR-AUDIT-03:** Failed authorization attempts shall be logged for security monitoring
- **SR-AUDIT-04:** Audit logs shall include minimum: timestamp, user ID, action type, resource affected, IP address, outcome
- **SR-AUDIT-05:** Logs shall be retained for minimum 90 days for security investigation purposes
- **SR-AUDIT-06:** System shall monitor for suspicious patterns (rapid failed logins, unusual access times, bulk data changes)
- **SR-AUDIT-07:** Audit log integrity shall be protected through database-level permissions restricting deletion

6.2.6 Additional Security Controls

- **SR-SEC-01:** CSRF tokens shall be implemented for all state-changing operations
- **SR-SEC-02:** Rate limiting shall be enforced (100 requests per minute per IP address)
- **SR-SEC-03:** Security headers shall be implemented (X-Frame-Options, X-Content-Type-Options, Content-Security-Policy)
- **SR-SEC-04:** Database principle of least privilege shall be enforced (application uses limited permissions account)
- **SR-SEC-05:** Regular security dependency updates shall be performed to patch known vulnerabilities
- **SR-SEC-06:** Output encoding shall be applied to all dynamic content to prevent XSS attacks

6.3 Usability Requirements

- Interface shall be intuitive and require minimal training
- System shall provide clear error messages and user feedback
- UI shall be responsive and work on desktop browsers (Chrome, Firefox, Safari, Edge)
- Color scheme shall maintain sufficient contrast for accessibility

6.4 Reliability Requirements

- System uptime shall be 99.5% during business hours
- Database backups shall be performed daily

- System shall recover gracefully from database connection failures

7. User Roles and Permissions

7.1 Administrator

Permissions:

- Full system access
- Create, read, update, and delete all products
- Manage user accounts
- View all audit logs
- System configuration access

7.2 Warehouse Manager

Permissions:

- Add new products
- View all inventory
- Update product quantities

- Edit product information
- View movement history for all products

7.3 Warehouse Staff

Permissions:

- View all inventory
- Update product quantities (IN, OUT, ADJUSTMENT)
- View movement history
- date product quantity
- DELETE /api/v1/products/:id - Delete product (Admin only)

Movement History:

- GET /api/v1/movements - Get all movements

8. System Constraints

8.1 Technical Constraints

- Must use Go programming language version 1.19 or higher
- Must use Fiber v2 web framework
- Must use PostgreSQL version 14 or higher
- Web interface must work on modern browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)

8.2 Business Constraints

- System must be deployed on university infrastructure
- Must comply with FERPA regulations for student data
- Development must follow ENPM680 course requirements and deadlines

9. Testing Requirements

9.1 Unit Testing

- All core business logic functions must have unit tests
- Minimum 70% code coverage required
- Use Go's built-in testing package

9.2 Integration Testing

- Test database connectivity and CRUD operations
- Test API endpoints with various inputs
- Verify authentication and authorization flows

9.3 User Acceptance Testing

- Verify all three functional requirements (FR-001, FR-002, FR-003)
- Test with different user roles
- Validate UI responsiveness and usability

9.4 Security Testing

- Test authentication bypass attempts
- Verify authorization controls for each role
- Test input validation with malicious payloads (SQL injection, XSS)
- Verify session management and timeout functionality
- Test audit logging completeness and integrity

10. Appendices

12.1 Glossary

- **Audit Log:** A chronological record of system activities and changes
- **Bcrypt:** Password hashing algorithm designed to be computationally expensive
- **CSRF:** Cross-Site Request Forgery - attack that forces users to execute unwanted actions
- **Fiber:** High-performance web framework for Go programming language
- **Movement:** Any change in product quantity (IN, OUT, or ADJUSTMENT)
- **SKU:** Stock Keeping Unit - unique identifier for each product
- **SQL Injection:** Attack technique that injects malicious SQL code to manipulate databases
- **XSS:** Cross-Site Scripting - injection of malicious scripts into web pages